Once you implement that behavior, you'll take out the pending line. RSpec will fail your specs if you forget—in effect, it's saying, "Hey, you said this wasn't working yet!"

Before moving on, let's hook our application up to a web server so we can actually see it working. Rack, the HTTP toolkit that Sinatra is built on top of, ships with a tool named rackup that makes it easy to run any Rack application (including apps built using Sinatra). We just need to define a rackup config file named config.ru with the following contents:

04-acceptance-specs/06/expense_tracker/config.ru

```
require_relative 'app/api'
run ExpenseTracker::API.new
```

Simple enough. We're just loading our application and telling Rack to run it. With that in place, we can boot our application by running rackup:

```
$ bundle exec rackup
[2017-06-13 13:34:10] INFO  WEBrick 1.3.1
[2017-06-13 13:34:10] INFO  ruby 2.4.1 (2017-03-22) [x86_64-darwin15]
[2017-06-13 13:34:10] INFO  WEBrick::HTTPServer#start: pid=45203 port=9292
```

While that's running, we can use a command-line tool like curl in another terminal window to send requests to our application.[10] The port=9292 bit from that last line tells us what port our application is running on, so let's send a request to localhost:9292:

```
$ curl localhost:9292/expenses/2017-06-10 -w "\n"
[]
```

It works! As expected, our application is responding with an empty JSON array.

Before moving on, commit the code into your favorite revision control system. That way, you'll easily be able to pick up where you left off. Grab a cup of coffee, try a couple of the exercises, and meet us in the next chapter.

Your Turn

In this chapter, we went over the major pieces of the software we're building. You used Bundler to manage all of your project's dependencies, including RSpec. You wrote your first spec to drive the app from its outermost layer, the HTTP interface, then wrote just enough Ruby code to get an idea of what the business logic needs to look like.

10. https://curl.haxx.se/